

22. Results

22.1 Headline Outcome

The tuned Random Forest classifies `risk_category` on a held-out set of 4,400 records with accuracy 0.6039, weighted F1 0.6006 and one-vs-rest ROC AUC 0.7461. Against a majority-class baseline of 0.4500, the model recovers a substantial share of the available signal.

Taken alone, sixty percent accuracy invites the reading that the model is mediocre. Section 22.2 argues that this reading is wrong, and that the correct comparison is not against 100% but against the ceiling the data itself permits.

Three further results follow: a secondary model that cannot work and the reason why, a class of defect in the serving path that no accuracy metric could have exposed, and a redundancy in the feature set that corrupts the explanations without touching the predictions.

22.2 The Model Operates at the Bayes Limit

`risk_category` is not a property of the synthetic individuals. It is a threshold applied to a continuous score which, by construction in `generate_dataset.py`, equals a weighted sum of five features plus Gaussian noise:

```
risk_score = -30·savings_rate + 25·(1 - credit_score/900)
             + 20·(min(dti,5)/5) - 15·(investment_experience/20)
             + 10·[emergency_fund < 3] + ε,   ε ~ N(0, 8)
```

Every one of those five quantities is available to the model as a feature. The deterministic component is therefore *exactly* recoverable in principle. What is not recoverable is ϵ , and ϵ has a standard deviation of 8.00 against a signal standard deviation of 7.642 — a signal-to-noise variance ratio of 0.912.

This permits the Bayes-optimal accuracy to be computed rather than guessed. For a record with signal s , the probability of each class is determined by where the two thresholds fall relative to s under the noise distribution:

```
P(Low | s) = Φ((t1 - s)/8)
P(Med | s) = Φ((t2 - s)/8) - Φ((t1 - s)/8)
P(High | s) = 1 - Φ((t2 - s)/8)
```

The thresholds were recovered numerically by solving for the values at which the induced class proportions equal the observed 35 / 45 / 20, giving $t_1 = -11.074$ and $t_2 = 2.598$. A Bayes-optimal classifier assigns the argmax of these three probabilities. Averaging its success probability across all 22,000 records yields the ceiling.

Quantity	Accuracy
Majority-class baseline	0.4500
Tuned Random Forest, held-out	0.6039
Bayes-optimal ceiling	0.6104

The model attains **98.9%** of the maximum accuracy achievable by any classifier on this label. The remaining 0.65 points are not a modelling deficiency; they are the price of a label that is roughly half noise.

Two independent observations corroborate that the Random Forest has converged on the Bayes decision rule rather than merely landing near its accuracy.

The predicted class distribution matches. The Bayes rule, faced with a noisy threshold, systematically retreats toward the central class: near either boundary the middle band is the safest guess, so *High* is predicted less often than it occurs. The model reproduces this almost exactly.

Class	True share	Bayes rule predicts	Random Forest predicts
Low	35.0%	32.9%	31.4%
Medium	45.0%	54.4%	56.2%
High	20.0%	12.7%	12.5%

The under-prediction of *High* is not a bias the model acquired from imbalanced data. It is the optimal response to threshold noise, and the Bayes rule commits it too.

Errors respect the ordinal structure. Of 4,400 test records, 2,657 are classified correctly and 1,706 of the 1,743 errors fall between adjacent bands. Only **37 records — 0.84% —** are misclassified across two bands, confusing *Low* with *High*.

	pred Low	pred Medium	pred High
true Low	906	617	17
true Medium	454	1373	153
true High	20	482	378

A model that had failed to learn the underlying score would scatter its errors. This one places them where the noise places them: at the thresholds.

The cost is borne by recall on the minority class. *High* is identified with precision 0.6898 but recall 0.4295 — the model is right when it says *High*, and says it too rarely. For a risk-screening application that asymmetry runs the wrong way, and Section 24 takes it up.

22.3 The Investment-Preference Model Cannot Work

The secondary classifier reports accuracy 0.2586, weighted F1 0.1892 and ROC AUC 0.4949 across five classes. It is beaten by the majority-class baseline of 0.2869, and its ROC AUC is indistinguishable from the 0.5000 of a coin.

The cause is in `generate_dataset.py`, at the point where the label is created:

```
def inv_pref(risk):
    if risk == 'Low':    return choice(opts, p=[0.40,0.25,0.10,0.20,0.05])
    elif risk == 'Medium': return choice(opts, p=[0.15,0.35,0.25,0.15,0.10])
    else:                return choice(opts, p=[0.05,0.20,0.35,0.10,0.30])
```

The draw is conditioned on `risk_category` and on nothing else. Age, income, investment experience, luxury spending — none of the twenty-six inputs enters. The label is a random sample from one of three fixed distributions, and it therefore carries no information about the features beyond what `risk_category` already encodes.

The ceiling follows directly. A classifier that knew each record's true risk band with certainty could do no better than always guess that band's modal preference:

```
ceiling | true risk =  $\sum_k P(\text{risk}=k) \cdot \max(p_k) = 0.3675$ 
```

And no classifier knows the true band; the best available estimate is 60.4% accurate. Propagating that gives an achievable ceiling of approximately **0.3210**.

Quantity	Accuracy
Random Forest achieved	0.2586
Majority-class baseline	0.2869
Ceiling given <i>predicted</i> risk	≈ 0.3210
Ceiling given <i>perfect</i> risk	0.3675

Even a perfect classifier would be wrong on nearly two records in three. The model's failure to beat the baseline is a separate and smaller problem — a five-class Random Forest on a label with no learnable structure will fit noise in the training partition and generalise worse than a constant — but the ceiling establishes that no amount of tuning, no alternative algorithm and no additional feature could rescue this target.

This is reported as a negative result. The system exposes `investment_preference` through its API and renders it in the interface, and this report does not claim the field is meaningful. The finding is about the construction of the label, not about behavioural finance, and it should not be read as evidence that investment preference is unpredictable in general.

22.4 Training–Serving Skew

The most consequential defects found in this project were invisible to every metric reported above, because they lay outside the code that produced those metrics.

Feature engineering was implemented twice: once in `preprocess.py`, which prepared the training data, and once in `inference.py`, which served live requests. The offline test set was evaluated through the first path. Live users were served through the second. The two disagreed in five places.

#	Feature	Training	Serving (as implemented)
1	<code>savings_rate</code>	fraction, -0.1 to 0.6	percentage, 0 to 100, passed through unscaled
2	<code>debt_to_income_ratio</code>	<code>debt / (income × 12)</code>	<code>debt / income</code> — 12× too large
3	<code>age_income_ratio</code>	<code>income / age</code>	<code>age / income</code> — inverted
4	ordinal maps	PhD , Middle , 4-level frequency	Doctorate , Medium , 5-level frequency
5	composite scores	min-max weighted, constants fitted on the training column	ad-hoc formulas invented in the serving module

The first is the most damaging, because `savings_rate` carries the largest SHAP attribution of any feature. A user reporting a 10% savings rate submitted the value `10`, which the model — trained on a column whose maximum is 0.6 — encountered sixteen standard deviations beyond anything in its experience. Decision trees do not extrapolate; every such value fell into the same terminal leaf.

The effect is visible in the model's response to that single input, holding all others fixed:

<code>savings_rate</code> submitted	Before the fix	After the fix, P(High)
0%	High (conf 0.534)	0.454
5%	—	0.438
10%	Medium (conf 0.472)	0.408
20%	—	0.345
30%	Medium (conf 0.507)	0.235
45%	—	0.152
60%	Medium (conf 0.456)	0.113
100%	Low (conf 0.466)	—

Before the repair, the predicted risk was not monotone in the savings rate and a user saving 10% of income was assessed *Medium* where the training scale placed them at *High*. After it, `P(High)` declines monotonically across the whole range, which is the behaviour the generating function requires.

The repair removed the possibility of recurrence rather than correcting the five instances. `preprocess.py` now writes every constant it fits — ordinal maps, IQR clip bounds, min-max ranges, imputation medians — to `models/preprocessing_params.json`, and `inference.py` loads that artefact instead of restating the values. No hardcoded encoding remains in the serving path.

Equivalence was then verified rather than assumed. Three hundred randomly drawn raw records were passed through the serving preprocessor and compared column by column against the rows the training preprocessor produced for the same records: all thirteen derived and encoded features agree to within 10^{-16} . Held-out records routed through the full serving path produce predictions identical to feeding the model its training-format features directly, on **every** row.

Re-running `preprocess.py` after the change left `processed_dataset.csv` byte-identical, so the trained model remained valid and no retraining was required. **The accuracy reported in Section 22.2 is now the accuracy a live user receives. Before the repair, it was not.**

The general lesson is worth stating. Every one of the five defects lived in the *agreement between two implementations of one computation*, and no test in the suite compared them. A test set drawn from the training pipeline validates the model; it says nothing about the pipeline that will serve it.

22.5 Redundancy Corrupts the Explanations

`savings_ratio` and `expense_ratio` are exact complements. Their definitions give `savings_ratio = 1 - expense_ratio` identically, and across the 22,000 records their Pearson correlation is -1.000000 , with `|savings_ratio + expense_ratio - 1|` never exceeding 4.4×10^{-16} . `savings_rate`, the raw column, is a third encoding of the same quantity — a noisy observation of `savings_ratio`, correlating with it at 0.944.

The global SHAP ranking splits the resulting attribution:

Rank	Feature	Mean abs. SHAP
1	savings_rate	0.0462
2	financial_discipline_score	0.0407
3	emergency_fund_months	0.0280
4	savings_ratio	0.0217
5	expense_ratio	0.0217

Ranks four and five are identical to four decimal places, which is what a tree ensemble produces when offered two perfectly anti-correlated columns: each split that might have used one is equally likely to use the other, and Shapley attribution divides between them. `financial_discipline_score` at rank two compounds the problem, since `savings_rate` is its highest-weighted component at 0.40.

A Random Forest is not harmed in accuracy by collinear inputs. The damage is confined to interpretation — and interpretation is the stated purpose of computing SHAP values at all. A reader taking the ranking at face value sees the savings signal distributed across three rows and two composites, and would substantially underestimate how completely one quantity dominates this model.

The defect was identified after the results were generated. Removing `expense_ratio` would leave accuracy essentially unchanged and make the explanation honest; doing so would require regenerating every figure in this report. It is recorded rather than silently corrected.

Figure 22.1 — *Confusion matrix heat-map*, 3×3, annotated with counts and row-normalised percentages. The off-diagonal mass sitting adjacent to the diagonal is the visual argument of Section 22.2. Place there.

Figure 22.2 — *Predicted-versus-true class distribution*, grouped bars for true / Bayes-rule / Random Forest across the three classes. This single figure carries the central claim of the chapter. Place in Section 22.2.

Figure 22.3 — *P(High) against submitted savings_rate*, two lines: before the fix (non-monotone) and after (monotone decreasing). Generate by calling `predict()` across a sweep. Place in Section 22.4.

Figure 22.4 — *Scatter of savings_ratio against expense_ratio*, showing the exact anti-diagonal. Shared with Figure 14.1; render once. Place in Section 22.5.

Table 22.1 — *Per-class precision, recall and F1* for the tuned Random Forest, with support. Values in Section 23.2. Place at the start of Section 22.2.